

REPO LEARNING DECK

dAstIII

A content distillation system: source monitoring, transcript extraction, AI summaries, evaluated quality, search, chat, highlights, and mobile reading.

RUST BACKEND

SVELTEKIT FRONTEND

OLLAMA-FIRST AI

S3 + LIBSQL + S3 VECTORS

SOURCES: README.MD; DOCS/INDEX.MD

CENTRAL THESIS

The app turns feed anxiety into a searchable, evaluated library.

The product watches sources, turns raw media or web text into readable artifacts, scores the AI output, and lets the user return through summaries, highlights, search, audio, or chat.

SOURCES: [README.MD](#); [DOCS/ARCHITECTURE/OVERVIEW.MD](#)

USER-FACING SURFACE

One workspace, several ways back to meaning.

Library

Channels, videos, saved searches, podcasts, and websites flow into one reading workspace.

Memory

Highlights, acknowledgements, vocabulary replacements, and conversations keep user intent separate from canonical content.

Retrieval

Keyword and semantic search feed both direct search results and grounded chat answers.

SOURCES: [DOCS/INDEX.MD](#); [DOCS/ARCHITECTURE/DATA-MODEL.MD](#)

The repository is split by runtime responsibility.

PATH	PURPOSE	WHAT TO READ FIRST
backend/	Rust Axum API, workers, stores, AI adapters.	backend/src/main.rs, backend/src/state.rs
frontend/	SvelteKit product UI, static build, browser auth.	frontend/src/routes/+page.svelte, frontend/src/lib/api.ts
docs/	VitePress technical docs.	docs/index.md
terraform/	Cloud Run, Firebase, AWS, IAM, secrets.	docs/operations/deployment.md
databricks/	Analytics bundle, SQL pipeline, dashboards.	databricks/databricks.yml

Only the backend owns durable changes and worker execution.

BROWSER	Uses SvelteKit product UI and VitePress docs UI.
FRONTEND	Calls the Rust backend directly with Firebase bearer tokens or trusted proxy headers.
BACKEND	Owns storage clients, local libSQL, API handlers, AI services, and long-running workers.
WORKERS	Advance queues, refresh feeds, evaluate summaries, and maintain search projections.

SOURCES: [DOCS/ARCHITECTURE/RUNTIME-TOPOLOGY.MD](#); [DOCS/ARCHITECTURE/FRONTEND-AND-API.MD](#)

Startup binds early, then assembles a single shared state object.

1. Load environment and runtime configs.
2. Initialize AWS, S3, S3 Vectors, and local libSQL.
3. Build service adapters for sources, AI, TTS, chat, and search.
4. Hydrate search progress and FTS when needed.
5. Spawn worker loops and bind API routes.

SOURCES: [BACKEND/SRC/MAIN.RS](#); [BACKEND/SRC/STATE.RS](#); [DOCS/ARCHITECTURE/RUNTIME-TOPOLOGY.MD](#)

AppState is the pressure point.

It carries the store, read cache, rate limiter, projection lock, FTS index, source services, AI services, chat service, mobile handoff sessions, cooldowns, and analytics client.

Canonical content is sacred. Search is rebuildable.

Channels, videos, transcripts, summaries, and video info are the source records. Search sources, chunks, keyword rows, and vector embeddings are projections that workers can recreate.

CANONICAL	USER OVERLAY	DERIVED
Channels, videos, transcripts, summaries, video info.	Subscriptions, acknowledgements, preferences, highlights, conversations.	Search sources, chunks, BM25 rows, embeddings, summary audio cache.

CONTENT PIPELINE

The expensive work moves in the background.

INPUT	YouTube, OpenAlex, podcast RSS, or website page.
CANONICALIZE	Resolve source, create metadata, and queue content states.
EXTRACT	Transcript or text extraction uses <code>summarize</code> , with <code>yt-dlp</code> <code>json3</code> fallback for timed captions.
DISTILL	Summaries are generated only after transcripts are ready, then evaluated by a separate model.
INDEX	Writes mark search work pending; the search worker chunks, embeds, and updates FTS later.

SOURCES: [DOCS/PIPELINES/CONTENT-PIPELINE.MD](#); [BACKEND/SRC/WORKERS/MOD.RS](#)

Model separation is part of the product contract.

ROLE	CONFIG	WHY IT MATTERS
Summarizer	OLLAMA_SUMMARY_MODEL	Creates and cleans content through Ollama /api/generate.
Evaluator	SUMMARY_EVALUATOR_MODEL	Must differ from the summarizer, and startup fails if they match.
Embedding	OLLAMA_EMBEDDING_MODEL	Powers semantic search when enabled; FTS continues without it.
Chat	OLLAMA_DEFAULT_CHAT_MODEL	Runs grounded conversations with retrieval budgets by intent.

SEARCH MODEL

Hybrid search is two retrieval legs plus rank fusion.

KEYWORD	libSQL FTS5 BM25 with title, section, source, channel, and timestamp metadata.
SEMANTIC	Optional S3 Vectors ANN or exact vector scan, backed by Ollama embeddings.
FUSION	Reciprocal Rank Fusion merges the legs; optional reranking can rescore top candidates.
SURFACE	Results group by video and keep one best match per source kind where possible.

SOURCES: [DOCS/PIPELINES/SEARCH-INDEXING.MD](#); [BACKEND/SRC/SERVICES/SEARCH/*](#)

Chat is retrieval planning, not a single search call.

The chat service classifies intent, expands queries across passes, retrieves keyword and semantic candidates, assembles context, then streams a response with sources.

SOURCES: DOCS/PIPELINES/AI-MODELS.MD;
BACKEND/SRC/SERVICES/CHAT/*

Intent budgets

- **Fact:** small source budget.
- **Comparison:** wider source budget.
- **Pattern:** widest standard budget.
- **Deep research:** raises the budget and pass count.

The SvelteKit app keeps transport logic centralized.

- Routes expose workspace, channel, queue, highlights, chat, vocabulary, login, logout, and mini reader.
- `frontend/src/lib/api.ts` owns product request helpers and cache invalidation.
- Generated bindings under `frontend/src/lib/bindings/` mirror backend payloads.
- Mobile shell pieces use shared components and safe-area aware UI.

Bootstrap pattern

The main workspace starts with `/api/workspace/bootstrap` so channel list, selected channel, AI status, search status, and initial snapshot arrive coherently.

SOURCES: `DOCS/ARCHITECTURE/FRONTEND-AND-API.MD`; `FRONTEND/SRC/ROUTES/*`; `FRONTEND/SRC/LIB/API.TS`

TRUST BOUNDARY

Every useful backend read resolves an access context.

Browser and Tauri clients normally call the Rust backend directly with Firebase ID tokens. Trusted first-party callers can use the proxy header path, but the backend still resolves scope before authorization.

FIREBASE BEARER TOKEN

PROXY AUTH HEADERS

ANONYMOUS SCOPED ACCESS

RATE LIMITS

SOURCES: [DOCS/ARCHITECTURE/FRONTEND-AND-API.MD](#); [BACKEND/SRC/SECURITY.RS](#); [BACKEND/SRC/FIREBASE_AUTH.RS](#)

MOBILE AND MINI

The mobile direction is not a wrapper only.

The repo includes a Tauri v2 Android shell, browser-auth handoff, native selection flows, mobile browse overlays, a mini reader route, and tests aimed at mobile navigation and viewport behavior.

src-tauri/

Android shell with opener and deep-link plugins.

/mini

Focused reading route with summary strips and swipe navigation.

E2E coverage

Mobile selection, shell viewport, workspace tabs, and navigation tests.

SOURCES: [DOCS/OPERATIONS/MOBILE-TAURI.MD](#); [SRC-TAURI/CARGO.TOML](#); [FRONTEND/SRC/LIB/COMPONENTS/MINI/*](#); [FRONTEND/E2E/*](#)

OPERATIONS

Local and production paths are deliberately different.

LOCAL

`./start_app.sh` runs frontend on 3543, backend on 3544, docs on 4173, and optional Android.

Shared env lives under `~/.config/dastill/`; AWS credentials should live outside repo env files.

Debug builds default semantic search on unless overridden.

PRODUCTION

Cloud Run serves backend; Firebase Hosting serves product frontend and docs.

Secret Manager holds sensitive payloads; Cloud Run env holds non-secret runtime config.

Release defaults semantic search off unless explicitly enabled.

SOURCES: [DOCS/OPERATIONS/LOCAL-DEVELOPMENT.MD](#); [DOCS/OPERATIONS/DEPLOYMENT.MD](#); [AGENTS.MD](#)

CURRENT POSTURE

This checkout is in maintenance runtime mode.

`.github/runtime-mode.env` currently sets `APP_RUNTIME_MODE=maintenance`. The docs say this builds the main product frontend in maintenance posture while keeping the backend path available for the mini experience.

SOURCES: `.GITHUB/RUNTIME-MODE.ENV`; `DOCS/OPERATIONS/DEPLOYMENT.MD`; `DOCS/OPERATIONS/LOCAL-DEVELOPMENT.MD`

The repo expects green tests before commits.

Backend

- `cargo check`
- `cargo test`
- `./scripts/cargo_audit.sh`

Frontend

- `bun run format:check`
- `bun run lint`
- `bun run check`
- `bun run test, test:e2e, build, bun audit`

ANALYTICS

Databricks is a separate analytics plane.

The backend can write analytics events to a Bronze table. The Databricks bundle defines a triggered pipeline that deduplicates Silver events and builds Gold views for processing, engagement, and summary consumption dashboards.

BRONZE	Backend-written app events.
SILVER	Typed, deduplicated app events in a private materialized view.
GOLD	Dashboard-facing processing, engagement, and consumption views.

SOURCES: [DATABRICKS/AGENTS.MD](#); [DATABRICKS/DATABRICKS.YML](#); [DATABRICKS/PIPELINES/*.SQL](#)

The system has clear invariants. Break them slowly.

- **Do not inline expensive work into writes.** Queue it, mark pending, let workers advance it.
- **Do not treat browser storage as canonical.** Scope every local cache by auth context.
- **Do not store secret payloads in Terraform state or repo files.** Terraform owns containers and IAM, not payloads.
- **Do not scale backend replicas casually.** In-process workers and local libSQL cache make one serving instance the current production assumption.
- **Do not make the evaluator the summarizer.** The startup guard exists to keep judgment independent.

Read the system from outside in, then from data inward.

1. `README.md` for the promise and setup shape.
2. `docs/architecture/overview.md` for components and repo layout.
3. `docs/architecture/runtime-topology.md` and `backend/src/main.rs` for process startup.
4. `docs/architecture/data-model.md` for canonical, user, and derived state.
5. `docs/pipelines/content-pipeline.md` and `backend/src/workers/` for asynchronous work.
6. `docs/pipelines/search-indexing.md` and `backend/src/services/search/` for retrieval.
7. `frontend/src/lib/api.ts` and `frontend/src/lib/components/workspace/` for product behavior.

Corpus covered in this deck.

- AGENTS.md, repo instructions, env strategy, verification rules.
- design.md, local visual system and frontend standards.
- README.md, product promise, setup, stack, model defaults.
- docs/index.md, docs navigation and capability overview.
- docs/architecture/*.md, system, runtime, API, data model.
- docs/pipelines/*.md, content, search, and AI behavior.
- docs/operations/*.md, local development and deployment.
- backend/src/main.rs, state.rs, workers, services, handlers.
- frontend/src/routes/*, frontend/src/lib/api.ts, components, tests.
- src-tauri/, Android shell manifests and source.
- .github/workflows/*.yml, validation and release behavior.
- databricks/, Asset Bundle, SQL pipeline, dashboard metadata.
- Recent git history from `git log -n 12` for direction signals.

External research was not used. Confidence is high for repo shape and documented behavior, medium for code paths not exhaustively traversed, and low for production runtime state outside this checkout.

OPEN QUESTIONS

The next pass should follow live data and failure paths.

- Which storage records dominate latency in a representative local startup?
- Which chat intents currently miss useful context in real conversations?
- Where does maintenance mode end for the main product UI, and what gates the return to live mode?
- Which Databricks dashboard metrics are actively used for product decisions?
- What source families beyond YouTube need the strongest ingestion hardening next?

SOURCES: INFERENCES FROM DOCS/OPERATIONS, DOCS/PIPELINES, BACKEND/SRC/SERVICES, DATABRICKS/